ITC 353 Project 1 Worksheet

NYC 311 Service Request Delay Prediction

Name: NYC 311 Analytics Student

Run NYC311_Analysis.R from this folder before running the question examples. Each numbered question below includes its answers, an executable R example, and output from the completed R run. The full script reproduces every table, chart, model, and metric. Original source files are preserved. All results describe the supplied April 2025 sample.

The R examples share objects created by the full script; run them in numerical order in the same R session. Full tables are in analysis/tables. Name is a permitted placeholder. Review the learning reflection in your own voice before submission.

Part 1: Business Understanding

1. Business Problem

In your own words, describe the business problem your team is trying to solve.

Response:

City operations leaders need to identify incoming requests that may take longer to close so they can plan follow-up and allocate limited attention. This project estimates recorded closure time and screens for unusually slow requests within comparable problem types. Recorded closure is an administrative outcome and does not establish that the underlying issue was fixed. The supplied sample contains only records marked Closed, which limits its representativeness for new incoming requests.

R code for question 1

```
cat("Goal: anticipate recorded closure delays at initial submission.\n")
```

Executed R output

```
Goal: anticipate recorded closure delays at initial submission.
```

2. Prediction Goal

What are you trying to predict in this project?

Linear regression model:

Predict log(1 + resolution_hours), where resolution_hours is the elapsed time from creation to recorded closure. Report validation RMSE on this transformed scale and separately check predictions converted to hours.

Logistic regression model:

Predict the probability that a request exceeds the training-set 75th percentile of closure time for its original problem type. The outcome is 1 for slow and 0 otherwise; small or unseen types use a global training threshold.

R code for question 2

```
print(business_goal)
```

Executed R output

```
                model                                          target
1   Linear regression              log(1 + hours to recorded closure)
2 Logistic regression Above training problem-specific 75th percentile
```

3. Business Use

How could city operations leaders use these predictions?

Response:

Use estimates to plan monitoring and follow-up queues, alongside urgency, service commitments, and staff judgment. The weak slow-request model should be evaluated in a monitored pilot before operational use. Its predictions should not determine which residents receive service.

R code for question 3

```
cat("Use: reviewed follow-up planning; validate before operational triage.\n")
```

Executed R output

```
Use: reviewed follow-up planning; validate before operational triage.
```

Part 2: Data Understanding and Exploratory Data Analysis

Before building models, explore the dataset carefully. Your goal in this section is to understand what the data contains, what problems need to be addressed, and which variables may be useful for prediction.

4. Initial Data Review

Record the basic structure of your dataset. You may not be able to complete the bottom half of the table right away. Return to it after you have finished inspecting the data, making cleaning decisions, and selecting the variables you will use for modeling.

Item	Answer
Number of rows before cleaning	40,000
Number of columns before cleaning	16
Number of rows after removing unusable records	39,668
Number of columns after selecting/creating variables for modeling	8 (6 predictors + 2 outcomes)

Briefly explain any major changes in the number of rows or columns.

Response:

The CSV has 40,000 data records and 16 columns; 40,001 counts the header as a row. Removing 332 records without closed timestamps leaves 39,668. No duplicate keys, invalid created timestamps, negative durations, or out-of-April created dates were removed. The exported model datasets have eight columns: six predictors and two outcomes. Categorical predictors expand into multiple regression coefficients.

R code for question 4

```
print(split_summary)
```

Executed R output

```
                                item value
1                           Raw rows 40000
2                       Raw columns    16
3                      Eligible rows 39668
4                      Training rows 27767
5                    Validation rows 11901
6                               Seed   353
7                 Predictor variables     6
8 Model columns including two outcomes     8
```

5. Missing Values and Data Quality

Create a missing values summary table for all columns. Note: this may be an iterative process. Recommendation: fill in the first three columns first, then complete the rest of the data exploration before making final decisions about removal or recoding.

Variable	Number Missing	Percent Missing	Decision	Reason
unique_key	0	0.000%	Remove predictor	Identifier; retain for audit
created_date	0	0.000%	Derive features	Known at submission
closed_date	332	0.830%	Target only	332 absent; exclude these rows
agency	0	0.000%	Remove predictor	Largely determined by problem
problem	0	0.000%	Keep and group	Pool rare training categories
problem_detail	429	1.073%	Remove predictor	High cardinality and overlap
additional_details	22,528	56.320%	Remove predictor	Incomplete free text
status	0	0.000%	Remove predictor	Post-submission; all Closed
location_type	4,532	11.330%	Remove predictor	Exact overlap with some problems
incident_zip	373	0.932%	Remove predictor	Use borough for parsimony
borough	0	0.000%	Keep	Five complete categories
community_board	0	0.000%	Remove predictor	Geographic overlap; many levels
open_data_channel_type	0	0.000%	Keep and recode	Rare level to training mode
due_date	39,766	99.415%	Remove predictor	99.415% missing; timing uncertain
resolution_description	148	0.370%	Remove predictor	Post-submission information
resolution_action_updated_date	0	0.000%	Remove predictor	Post-submission information

Based on your missing values summary, answer the following:

Which variables have the most missing data?

due_date is missing for 39,766 records (99.415%); additional_details for 22,528 (56.320%); location_type for 4,532 (11.330%). All 16 columns appear in the table.

Are any variables too incomplete to be useful? Explain.

due_date is almost entirely absent and its availability at initial submission is not established, so it is excluded. additional_details is an incomplete free-text field and is omitted from this introductory model. Missing location_type could be encoded, but the field is ultimately excluded because some levels exactly duplicate problem types in the training design matrix.

Are there missing values that could be meaningful? For example, does a blank value sometimes mean "not applicable" rather than an error?

A blank detail or location field may reflect an issue for which that field is not applicable, or inconsistent collection. The file cannot distinguish these explanations. Missing closed_date does not mean zero hours: 332 such records are labeled Closed, but their duration cannot be computed.

R code for question 5

```
print(missingness, row.names = FALSE)
```

Executed R output

```
                     variable missing percent
                   unique_key       0  0.0000
                 created_date       0  0.0000
                  closed_date     332  0.8300
                       agency       0  0.0000
                      problem       0  0.0000
                problem_detail     429  1.0725
            additional_details   22528 56.3200
                       status       0  0.0000
                location_type    4532 11.3300
                 incident_zip     373  0.9325
                      borough       0  0.0000
               community_board       0  0.0000
       open_data_channel_type       0  0.0000
                     due_date   39766 99.4150
       resolution_description     148  0.3700
resolution_action_updated_date       0  0.0000
```

6. Categorical Variable Distributions

Create frequency tables or bar charts for the following variables: agency, problem, problem_detail, location_type, borough, community_board, and open_data_channel_type. For each variable, focus on the most common categories.

Variable	Most common categories	Rare categories under 100	Modeling decision
agency	NYPD (18,997); HPD (7,445); DSNY (4,331)	2 of 14	Remove predictor
problem	Illegal Parking (7,088); Noise - Residential (4,194); HEAT/HOT WATER (2,479)	84 of 142	Keep and group
problem_detail	Loud Music/Party (4,519); Blocked Hydrant (2,079); ENTIRE BUILDING (1,672)	529 of 598	Remove predictor
location_type	Street/Sidewalk (13,365); RESIDENTIAL BUILDING (7,445); Residential Building/House (4,623)	66 of 83	Remove predictor
borough	BROOKLYN (12,603); QUEENS (9,692); MANHATTAN (8,557)	0 of 5	Keep
community_board	12 MANHATTAN (1,168); 01 BROOKLYN (1,078); 12 BRONX (1,044)	13 of 75	Remove predictor
open_data_channel_type	ONLINE (17,113); PHONE (11,166); MOBILE (8,627)	1 of 5	Keep and recode

Which categorical variables seem most useful for predicting resolution time? Why?

Problem type should capture differences in agency workflow and task complexity. Borough, submission channel, and creation time may add context. Agency is descriptively informative but is largely determined by problem type, making separate causal interpretations inappropriate.

Which categorical variables may be difficult to use because they have too many categories or too many rare values?

There are 142 problem types and 598 problem-detail categories including Missing; 84 and 529 respectively have fewer than 100 observations. Location type has 83 categories and community board 75. Sparse categories increase instability and make unseen validation levels difficult.

Did your team decide to group, remove, or simplify any categorical variables? Explain.

Problem categories with fewer than 100 training records are pooled as Other. Borough is retained. The single rare channel is mapped to the training modal channel ONLINE because its combined rare group is too small to estimate. Future unseen levels use the saved training mapping. Agency, problem_detail, location_type, community_board, and ZIP are omitted for redundancy, sparsity, or parsimony. All mappings are learned on training rows only.

R code for question 6

```
print(category_summary, row.names = FALSE)
for (v in cats) {
  cat("\n", v, "\n")
  print(head(sort(table(raw[[v]], useNA="ifany"),
                  decreasing=TRUE), 3))
}
```

Executed R output

```
              variable levels rare_under_100
                agency     14              2
               problem    142             84
        problem_detail    598            529
         location_type     83             66
               borough      5              0
        community_board     75             13
 open_data_channel_type      5              1
 agency
 NYPD   HPD  DSNY
18997  7445  4331
 problem
    Illegal Parking Noise - Residential      HEAT/HOT WATER
                7088                4194                2479
 problem_detail
Loud Music/Party  Blocked Hydrant  ENTIRE BUILDING
            4519             2079             1672
 location_type
           Street/Sidewalk       RESIDENTIAL BUILDING
                      13365                       7445
Residential Building/House
                       4623
 borough
 BROOKLYN    QUEENS MANHATTAN
    12603      9692      8557
 community_board
12 MANHATTAN  01 BROOKLYN     12 BRONX
        1168         1078         1044
 open_data_channel_type
ONLINE  PHONE MOBILE
 17113  11166   8627
```

7. Date and Time Patterns

Use created_date to explore when service requests were submitted. Create at least two of the following:

- Requests by day of the month
- Requests by day of week
- Requests by hour of day
- Requests on weekdays versus weekends

Are service requests evenly distributed across the month?

Counts are uneven: April 25 has 1,541 records, April 30 only 361, and April 1 has 959. This is a supplied sample, not a verified complete month of city demand. The boundary-day differences may reflect sample coverage and should not be interpreted as actual citywide demand changes.

Do requests appear more common on certain days of the week or times of day?

Tuesday has the most total records (6,770), but April contains five Tuesdays. After dividing by calendar-day counts, Thursday averages 1,429.75 requests per day and Wednesday 1,244.80. The peak hour is 09:00 with 2,325 requests; 04:00 has 453. These are descriptive sample patterns.

What date/time features did your team create for modeling? Why might they be useful?

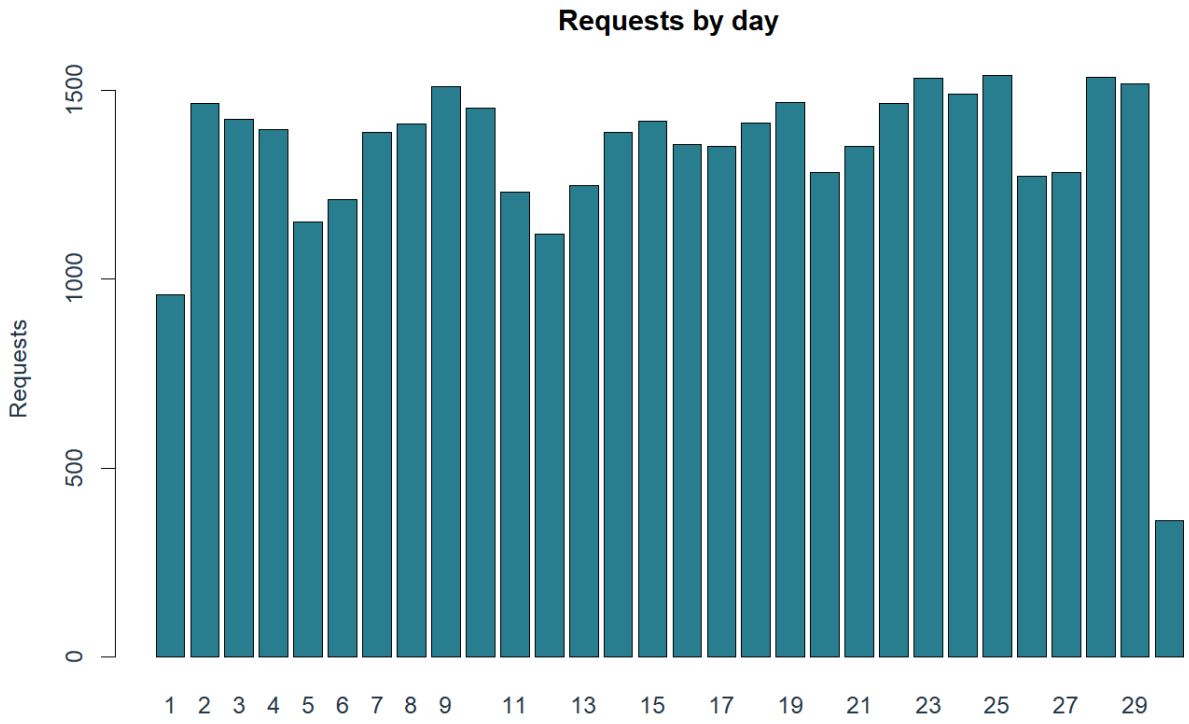
Created weekday is a seven-level factor. Hour is encoded by sine and cosine so 23:00 and 00:00 remain adjacent. Day of month and weekend are also created for EDA. They are not final predictors: weekday already expresses weekend membership, and a single-month day trend may not generalize.

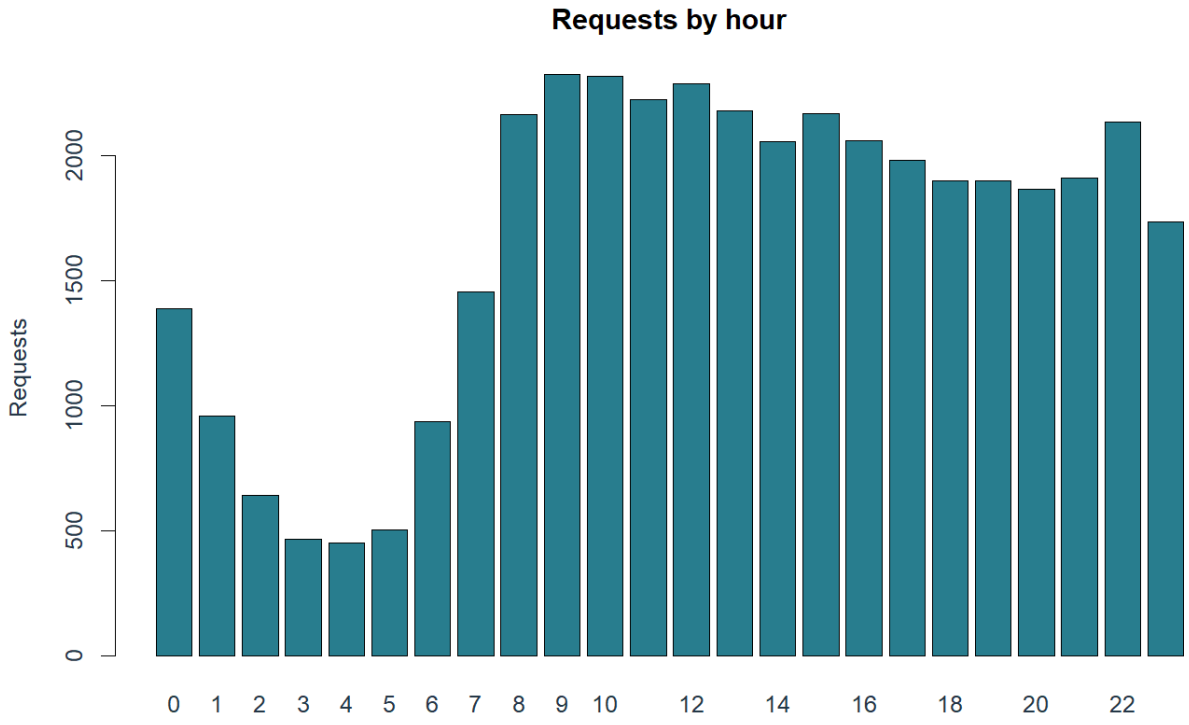
R code for question 7

```
print(read.csv("analysis/tables/weekday_adjusted.csv"))
print(range(table(d$day)))
print(sort(table(d$hour), decreasing=TRUE)[c(1,24)])
```

Executed R output

```
  weekday requests calendar_days per_day
1     Mon     5667             4 1416.75
2     Tue     6770             5 1354.00
3     Wed     6224             5 1244.80
4     Thu     5719             4 1429.75
5     Fri     5581             4 1395.25
6     Sat     5013             4 1253.25
7     Sun     5026             4 1256.50
[1]  361 1541
   9    4
2325  453
```





8. Resolution Time Distribution

Create a resolution_hours variable using created_date and closed_date. Then create the following:

1. A summary table for resolution_hours
2. A histogram of resolution_hours
3. A histogram of log_resolution_hours

Use this table to record your summary statistics.

Statistic	Value in hours unless count
Minimum	0
Mean	251.3556
Median	6.6817
75th percentile	66.0012
90th percentile	437.4729
95th percentile	1,251.4371
Maximum	9,888.1831
Zero hours	677
Over 30 days	2,930

What does the histogram of raw resolution_hours show?

The raw distribution is strongly right-skewed: median 6.68 hours versus mean 251.36 hours. A small fraction of very long cases stretches the horizontal axis. The 95th percentile is 1,251.44 hours and the maximum 9,888.18 hours.

Why might log_resolution_hours be useful for the linear regression model?

log1p(resolution_hours) compresses the upper tail and retains valid zero-hour records. Linear regression then minimizes squared error in log(1 + hours), emphasizing relative differences. Better log-scale fit does not guarantee accurate hour-scale forecasts.

Are there very short or very long resolution times? How did your team decide to handle them?

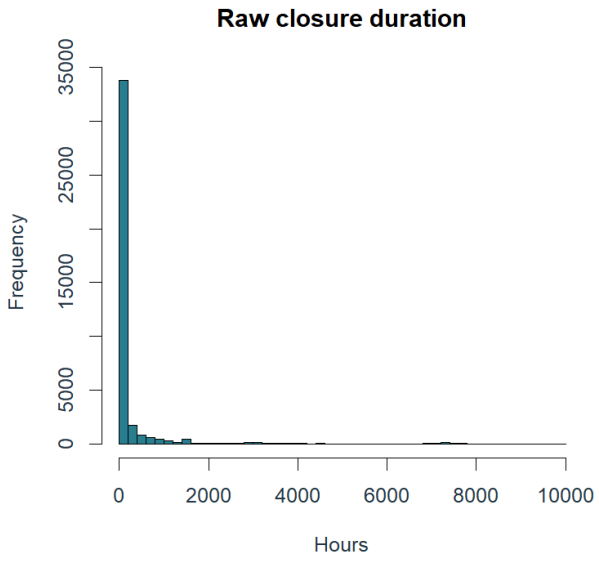
There are 677 zero-hour records and 2,930 records over 30 days. Both are retained because equal timestamps and long durations are possible and cannot be proven erroneous from this file. Their interpretation requires caution. The log transformation reduces their leverage; negative durations would be excluded, but none occur.

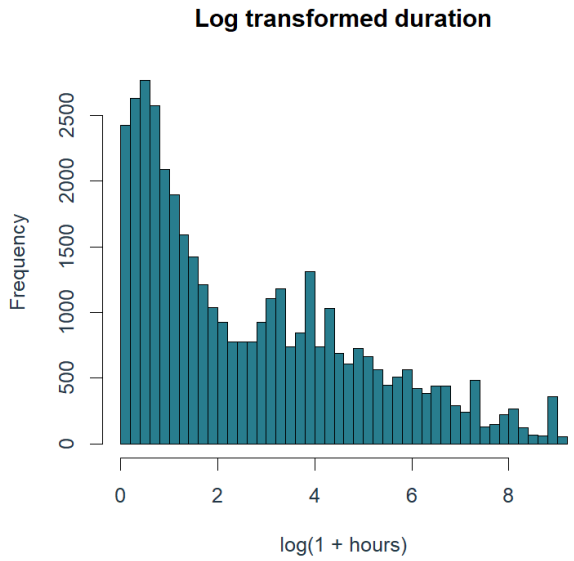
R code for question 8

```
print(duration_summary, row.names=FALSE)
hist(clean$resolution_hours, breaks=60)
hist(clean$log_resolution_hours, breaks=50)
```

Executed R output

```
       statistic      value
         Minimum    0.00000
            Mean  251.35559
          Median    6.68167
 75th percentile   66.00118
 90th percentile  437.47289
 95th percentile 1251.43711
         Maximum 9888.18306
      Zero hours  677.00000
    Over 30 days 2930.00000
```





9. Resolution Time by Key Variables

Explore how resolution time differs across important categories. Create at least three of the following:

- Median resolution time by agency
- Median resolution time by problem
- Median resolution time by borough
- Median resolution time by open_data_channel_type
- Boxplot of resolution time by agency
- Boxplot of resolution time by borough

Record key findings below.

Variable	Comparison	Pattern	Model implication
agency	Eligible group medians	EDC 7,254.84 h; NYPD 1.43 h	Large case-mix differences
problem	Eligible types with n >= 100	Helicopter 7,254.84 h; Rodent 0 h	Retain problem; interpret closure cautiously
borough	Median duration	Staten Island 13.07 h; Bronx 10.82 h	Context, not causal ranking
channel	Median duration	UNKNOWN 45.02 h; PHONE 25.97 h	May reflect different issue mix

Which agencies or problem types tend to have longer resolution times?

Among agencies with at least 100 eligible records, EDC has median 7,254.84 hours (n = 251), TLC 1,213.84 (n = 409), and DOB 293.82 (n = 1,202). Noise - Helicopter has median 7,254.84 hours and Building/Use 2,488.14 hours. These are recorded closure patterns, not an agency performance ranking adjusted for case mix.

Which agencies or problem types tend to be resolved quickly?

NYPD has median 1.43 hours across 18,997 requests. Rodent has median zero hours across 450 records; that is a timestamp observation, not evidence of instantaneous pest removal. Small categories are not used to establish reliable rankings.

Why might it be unfair to use one single definition of “slow” for every type of service request?

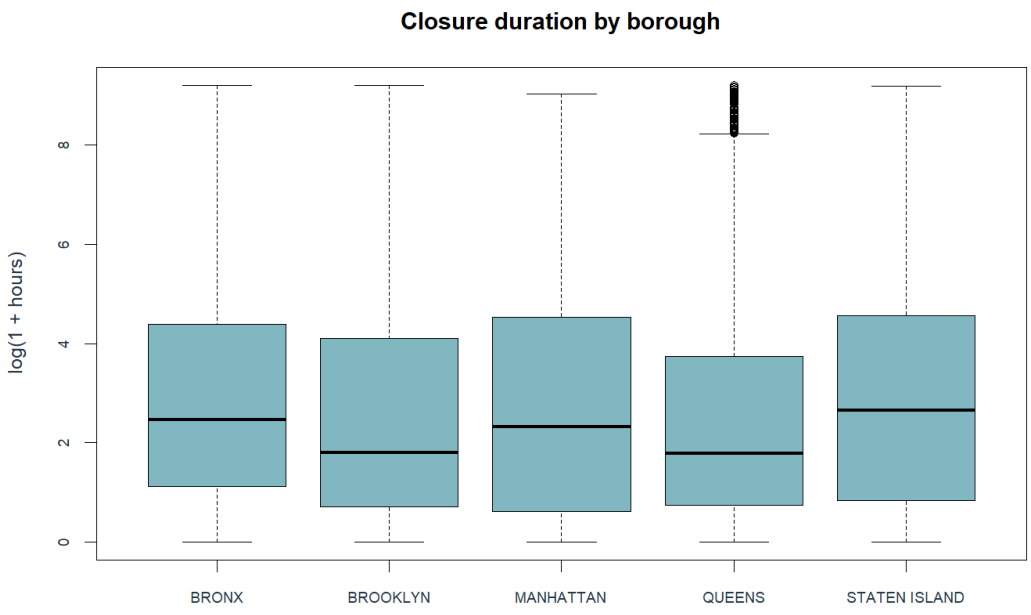
Different problem types require different actions and have different recorded workflows. A universal threshold would label naturally lengthy work slow even when it is typical for that problem. A within-problem percentile creates a relative comparison, although it is not an official service-level deadline.

R code for question 9

```
for (v in c("agency", "problem", "borough",
            "open_data_channel_type")) {
  z <- read.csv(paste0("analysis/tables/duration_by_",v,".csv"))
  cat("\n",v," (n >= 100)\n")
  print(head(z[z$n >= 100,], 4), row.names=FALSE)
}
```

Executed R output

```
 agency  (n >= 100)
 category    n   median     mean
      EDC  251 7254.840 7271.468
      TLC  409 1213.840 1585.513
      DOB 1202  293.822 1477.036
      DPR  990  116.717  929.056
 problem  (n >= 100)
          category   n  median    mean
 Noise - Helicopter 251 7254.84 7271.47
       Building/Use 240 2488.14 3507.83
     Taxi Complaint 106 1923.03 2256.51
 Food Establishment 158 1447.96 1431.80
 borough  (n >= 100)
      category     n   median    mean
 STATEN ISLAND  1505 13.07333 266.573
         BRONX  7623 10.82361 188.017
     MANHATTAN  8316  9.16472 369.333
      BROOKLYN 12552  5.08722 206.019
 open_data_channel_type  (n >= 100)
 category     n   median     mean
  UNKNOWN  3093 45.01667 732.5458
    PHONE 11098 25.96667 244.2106
   ONLINE 17015  5.21111 263.4668
   MOBILE  8461  2.25000  60.4951
```



10. Relationships Between Important Variables

Create at least two cross-tabulations to explore relationships between categorical variables. Recommended cross-tabs: agency by problem, problem by borough, problem by open_data_channel_type, and borough by open_data_channel_type.

Cross-tab	Relationship	Finding
Agency by problem	Routing of problem types	138 of 142 types occur within one agency
Borough by channel	Within-borough channel shares	Phone: Bronx 34.66%, Manhattan 24.37%
Problem by borough	Request mix across geography	Full counts and row percentages exported

Are some problem types handled mostly by specific agencies?

Yes. In the supplied sample, 138 of 142 problem types occur entirely within one agency. The full agency-by-problem table is saved, and a compact excerpt is included below. Agency and problem should therefore not be interpreted as independent drivers.

Do some types of requests appear more common in certain boroughs or submission channels?

Submission channel mix differs by borough. Phone accounts for 34.66% of Bronx records versus 24.37% of Manhattan records; online accounts for 36.69% versus 46.98%. These proportions describe recorded users and request mix, not all residents or access needs. The problem-by-borough table is also saved for inspection.

How could these relationships affect your model results or interpretation?

Correlated predictors can make individual coefficients unstable or redundant. Channel and borough associations may reflect differences in problem mix and reporting behavior. I retain problem but omit agency; I also remove location_type after its training-only design check finds exact redundancy. Model coefficients are associations, not causal effects.

R code for question 10

```
ap <- table(raw$problem,raw$agency)
cat("Single-agency problem types:",
    sum(rowSums(ap>0)==1), "of", nrow(ap), "\n")
print(table(raw$agency,raw$problem)[
  c("NYPD","HPD","EDC"),
  c("Illegal Parking","HEAT/HOT WATER","Noise - Helicopter")])
print(round(100*prop.table(
  table(raw$borough,raw$open_data_channel_type),1),2))
```

Executed R output

```
Single-agency problem types: 138 of 142

      Illegal Parking HEAT/HOT WATER Noise - Helicopter
  NYPD            7088              0                  0
  HPD                0           2479                  0
  EDC                0              0                251

               MOBILE ONLINE OTHER PHONE UNKNOWN
  BRONX         23.38  36.69  0.00 34.66    5.27
  BROOKLYN      22.30  42.74  0.00 26.83    8.13
  MANHATTAN     21.50  46.98  0.01 24.37    7.14
  QUEENS        20.69  43.58  0.00 26.97    8.76
  STATEN ISLAND 12.27  45.09  0.00 29.05   13.59
```

11. EDA Summary

Based on your exploratory analysis, summarize the most important things your team learned before modeling.

Key EDA finding	Why it matters
Strongly skewed duration	Median 6.68 h; mean 251.36 h. Use log1p and separate hour metrics.
332 missing closed dates	Cannot calculate an observed target; exclude without imputation.
Problem overlaps agency and location	Use a parsimonious predictor set; check training design rank.

Which variables do you expect to be most useful for prediction? Explain.

Problem is expected to be most informative for absolute closure duration; borough, channel, weekday, and cyclic hour may add information. For relative slow status, the problem-based definition removes much of the between-problem signal, so good duration prediction need not imply good slow-request classification.

What is one data issue your team will need to address before modeling?

The 332 Closed records without closed_date cannot provide a target. They are removed without inventing outcomes. More broadly, all statuses are Closed, so the sample may exclude still-open requests and underestimate delays among all incoming requests.

R code for question 11

```
cat("Median hours:",median(clean$resolution_hours),"\n")
cat("Mean hours:",mean(clean$resolution_hours),"\n")
cat("Closed status without closed_date:",
    sum(raw$status=="Closed" & is.na(raw$closed_date)),"\n")
```

Executed R output

```
Median hours: 6.68167
Mean hours: 251.356
Closed status without closed_date: 332
```

Part 3: Data Preparation

12. Resolution Time

How did your team calculate resolution time?

Formula or explanation:

Parse created_date and closed_date as NYC local timestamps, then calculate as.numeric(difftime(closed, created, units = "hours")). April has no daylight-saving transition. This measures time to recorded closure; it is not a direct measure of resident satisfaction or confirmed problem resolution.

What unit did you use?

[ ] Minutes [X] Hours [ ] Days [ ] Other

R code for question 12

```
resolution_hours <- as.numeric(difftime(
  d$closed,d$created,units="hours"))
print(head(resolution_hours))
```

Executed R output

```
[1]    0.646111    0.592778 3811.809167   22.127500    8.215833    4.956667
```

13. Records Removed

What records did your team remove before modeling?

Records removed	Reason
332 with missing closed_date	Cannot compute observed closure duration
0 duplicate keys	No repeated unique_key values
0 invalid created dates, negative durations, or outside-April rows	Checks implemented; no additional exclusions

R code for question 13

```
print(table(reason))
cat("Duplicate keys:",sum(duplicated(raw$unique_key)),"\n")
cat("Negative durations:",
    sum(d$resolution_hours<0,na.rm=TRUE),"\n")
```

Executed R output

```
reason
Missing or invalid closed date                       Retained
                           332                          39668
Duplicate keys: 0
Negative durations: 0
```

14. New Variables Created

List the new variables your team created.

New variable	How created	Purpose
created, closed	Parse local timestamp strings	Calculate elapsed time
resolution_hours	difftime(closed, created), hours	Raw target and EDA
log_resolution_hours	log1p(resolution_hours)	Linear regression outcome
day, hour, weekday	Extract creation components	EDA and time features
weekend	Saturday or Sunday indicator	EDA only; redundant with weekday
hour_sin, hour_cos	sin/cos(2*pi*hour/24)	Cyclic hourly predictors
slow	Hours exceed frozen training threshold	Binary logistic outcome
Grouped categorical values	Training counts and saved mapping	Stable sparse-level handling

R code for question 14

```
print(head(clean[,c("day","hour","weekday","weekend",
  "hour_sin","hour_cos","resolution_hours",
  "log_resolution_hours")]),row.names=FALSE)
```

Executed R output

```
 day hour weekday weekend  hour_sin     hour_cos resolution_hours
  26   22     Sat       1 -0.500000  8.66025e-01         0.646111
  20   18     Sun       1 -1.000000 -1.83691e-16         0.592778
  27   15     Sun       1 -0.707107 -7.07107e-01      3811.809167
   2   15     Wed       0 -0.707107 -7.07107e-01        22.127500
  27   22     Sun       1 -0.500000  8.66025e-01         8.215833
  11   21     Fri       0 -0.707107  7.07107e-01         4.956667
 log_resolution_hours
             0.498416
             0.465480
             8.246122
             3.141022
             2.220923
             1.784511
```

15. Missing Values

How did your team handle missing values?

Field	Decision	Reason
closed_date	Remove 332 rows	Do not impute the target
location_type	Explore as Missing; omit predictor	Exact training-design redundancy
problem_detail, additional_details, incident_zip, due_date	Omit these predictors	Sparsity, complexity, geography overlap, or timing
Final model predictors	No missing values after mappings	Verified in R

R code for question 15

```
cat("Missing final training predictors:",
    sum(is.na(train[,predictors])),"\n")
cat("Missing final validation predictors:",
    sum(is.na(validation[,predictors])),"\n")
```

Executed R output

```
Missing final training predictors: 0
Missing final validation predictors: 0
```

16. Categorical Variables

Which categorical variables did your team use in the models?

Categorical variable	Handling	Reason
problem	Training n < 100 pooled as Other	Limit sparse coefficients
borough	Retain all five levels	Coarse location context
open_data_channel_type	Keep common levels; rare/unseen to saved fallback	Single rare channel cannot support its own effect
weekday	Seven-level factor, Monday reference	Allow weekday differences

R code for question 16

```
for (v in c("problem","borough","open_data_channel_type",
            "weekday")) {
 cat(v,":",nlevels(train[[v]]),"model levels\n")
}
print(maps$open_data_channel_type)
```

Executed R output

```
problem : 47 model levels
borough : 5 model levels
open_data_channel_type : 4 model levels
weekday : 7 model levels
$keep
[1] "MOBILE"  "ONLINE"  "PHONE"   "UNKNOWN"
$levels
[1] "ONLINE"  "MOBILE"  "PHONE"   "UNKNOWN"
$fallback
[1] "ONLINE"
```

Part 4: Data Leakage Check

A model should only use information that would be available when the service request is first submitted.

17. Leakage Variables

Identify any variables that should not be used as predictors because they would create data leakage.

Variable	Why excluded from predictors
closed_date	Unknown when a request first arrives
status	Snapshot reflects later progress; all Closed here
resolution_description	Records agency action after submission
resolution_action_updated_date	Timestamp of later action
resolution_hours, log_resolution_hours, slow	Outcomes derived from future closure; never predictors
due_date	Availability at submission unverified; almost entirely missing

R code for question 17

```
leakage <- c("closed_date","status","resolution_description",
 "resolution_action_updated_date","resolution_hours",
 "log_resolution_hours","slow")
stopifnot(!any(predictors %in% leakage))
cat("Leakage predictor check passed.\n")
```

Executed R output

```
Leakage predictor check passed.
```

18. Final Predictor Set

List the variables your team used as predictors in your final models.

Predictor	Linear	Logistic	Reason
problem	Yes	Yes	Issue/workflow context
borough	Yes	Yes	Coarse geography
open_data_channel_type	Yes	Yes	Submission context
weekday	Yes	Yes	Day-of-week patterns
hour_sin	Yes	Yes	Cyclic time of submission
hour_cos	Yes	Yes	Completes cyclic representation

R code for question 18

```
print(predictors)
print(formula(linear))
print(formula(logistic))
```

Executed R output

```
[1] "problem"                "borough"                "open_data_channel_type"
[4] "weekday"                "hour_sin"               "hour_cos"
log_resolution_hours ~ problem + borough + open_data_channel_type +
    weekday + hour_sin + hour_cos
slow ~ problem + borough + open_data_channel_type + weekday +
    hour_sin + hour_cos
```

Part 5: Holdout Validation

19. Training and Validation Split

In this project, you will use the holdout method to evaluate your models. The holdout method divides the data into two independent parts:

- The training set is used to build the model.

- The validation set is used to evaluate how well the model predicts new, unseen observations.

Record how your team split the data.

Item	Answer
Percent used for training	70% (27,767 / 39,668)
Percent used for validation	30% (11,901 / 39,668)
Training observations	27,767
Validation observations	11,901
Random seed	353

Why did your team use this split?

A reproducible 70/30 random holdout provides 27,767 training rows and 11,901 validation rows, using seed 353. It leaves enough examples for estimating categorical effects and a substantial unseen evaluation set. The split is performed before rare-level mappings and slow thresholds are estimated.

Why is it important to evaluate model performance on the validation set instead of only using the training set?

Training performance is optimistic because the same observations estimated the coefficients. Validation measures performance on held-out records. This random within-April split still does not test future months or dependence between related requests; a later-month test is needed before deployment.

R code for question 19

```
print(table(ifelse(seq_len(nrow(clean)) %in% train_id,
                   "Training","Validation")))
cat("Seed: 353; training fraction:",
    nrow(train)/nrow(clean),"\n")
stopifnot(!any(train$unique_key %in% validation$unique_key))
```

Executed R output

```
  Training Validation
     27767      11901
Seed: 353; training fraction: 0.699985
```

Part 6: Linear Regression Model

20. Linear Regression Outcome

What outcome variable did you use for the linear regression model?

log_resolution_hours = log1p(resolution_hours).

Did you transform the outcome variable?

[ ] No [X] Yes - explain the transformation:

Yes. I used the natural logarithm of 1 plus hours, which is defined at zero and reduces the influence of the long upper tail.

Why was this outcome appropriate for predicting resolution time?

Closure duration is nonnegative and extremely skewed. The transformation supports modeling broad relative duration patterns while keeping zero-hour records. I also evaluate a training-residual smearing conversion to hours to avoid presenting transformed performance as an hour-scale error.

R code for question 20

```
train$log_resolution_hours <- log1p(train$resolution_hours)
print(summary(train$log_resolution_hours))
```

Executed R output

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.000   0.758   2.023   2.692   4.217   9.177
```

21. Linear Regression Validation Performance

Build the linear regression model using the training set. Then use the model to predict the outcome for the validation set.

Record your model performance.

Metric on log1p-hours scale	Training	Validation
RMSE	1.2938	1.3089
MAE	0.9156	0.9278
R_squared	0.6699	0.6582

Use RMSE as your primary performance measure. A lower RMSE means the model's predictions are closer to the actual values.

What does the validation RMSE tell you about the model's predictive performance?

Validation RMSE is 1.3089 log1p-hours versus 2.2388 for predicting the training mean log duration. Validation MAE is 0.9278 and predictive R-squared is 0.6582. RMSE is not 1.31 hours. On the hour scale, smearing-adjusted RMSE is 1,373.18 hours, worse than the training-mean baseline of 970.97 hours.

Is there a large difference between the training RMSE and validation RMSE? What might that suggest?

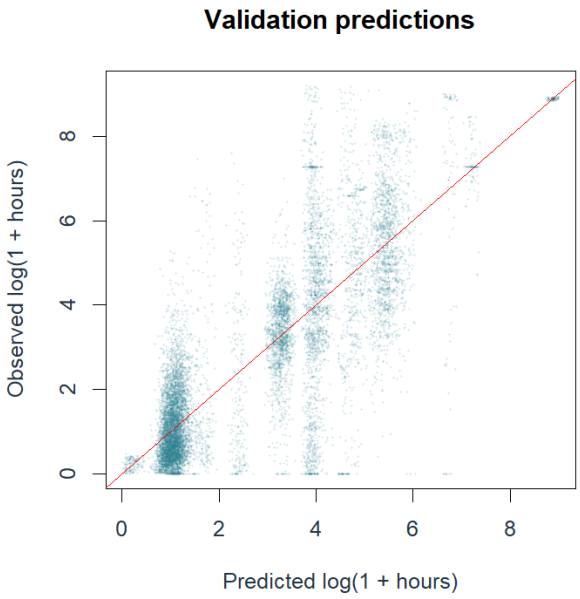
Training RMSE is 1.2938 and validation RMSE is 1.3089, a small increase of 0.0151. This does not suggest a large overfitting gap on this split, but it does not establish future-month reliability or eliminate sampling bias.

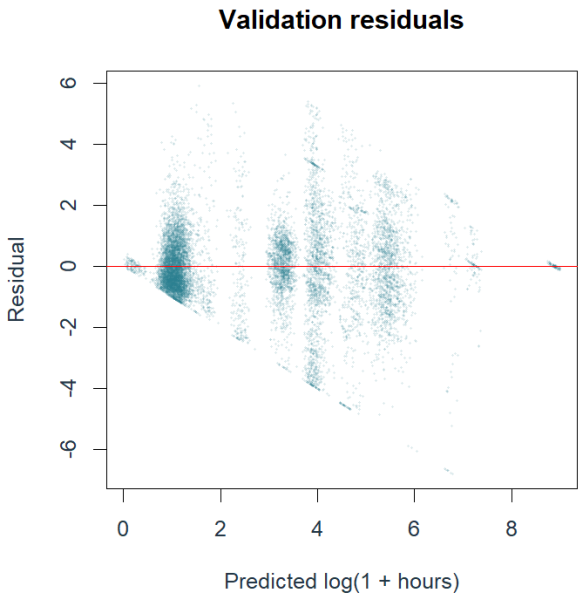
R code for question 21

```
linear <- lm(reformulate(predictors,
                         "log_resolution_hours"), data=train)
pred_valid <- predict(linear, newdata=validation)
sqrt(mean((validation$log_resolution_hours-pred_valid)^2))
print(linear_metrics,row.names=FALSE)
print(hours_metrics,row.names=FALSE)
```

Executed R output

```
        set   RMSE      MAE R_squared
   Training 1.2938 0.915567  0.669945
 Validation 1.3089 0.927796  0.658189
                    model     RMSE     MAE    R_squared
 Smearing adjusted linear 1373.178 316.646 -1.00009e+00
   Training mean baseline  970.967 392.307 -1.31544e-05
```





22. Linear Regression Interpretation

What did the linear regression model tell you?

The model explains substantial variation on the transformed scale using problem type and submission context. Validation predictive R-squared is 0.6582, and log-scale RMSE improves over the intercept-only baseline. Coefficients and residual plots are saved for review; the residual spread and hour-scale errors show substantial remaining uncertainty.

Was the model useful from a business perspective? Why or why not?

It may support broad planning about relative closure duration, but it is not reliable for individual completion-time promises. The smearing-adjusted hour-scale RMSE is worse than a simple mean baseline, even though hour-scale MAE improves from 392.31 to 316.65 hours. Large cases dominate squared error.

R code for question 22

```
print(read.csv("analysis/tables/linear_baseline_log.csv"))
cat("Training residual smearing factor:",smearing,"\n")
```

Executed R output

```
    RMSE     MAE    R_squared
1 2.2388 1.88367 -5.65145e-06
Training residual smearing factor: 2.96069
```

Part 7: Logistic Regression Model

23. Defining a Slow Request

For the logistic regression model, the response variable should be binary:

- 1 = slow request, the target class
- 0 = not slow request, the nontarget class

How did your team define a “slow” service request?

A request is slow if its duration is strictly greater than the training-set 75th percentile for its original problem type (R quantile type 7). If a problem has fewer than 30 training records or was unseen in training, use the global training 75th percentile. Validation labels use these frozen training thresholds; ties are not slow.

Why is this definition reasonable for the business problem?

This compares requests with broadly similar work and uses only training data to set benchmarks. The threshold is a relative historical benchmark, not an official SLA. Ties and the global fallback mean the slow fraction is not exactly 25%: training is 25.21% and validation 26.17%. Small categories have a less comparable fallback.

R code for question 23

```
print(head(read.csv("analysis/tables/slow_thresholds.csv")))
cat("Global training q75 hours:",global_q75,"\n")
cat("Training slow rate:",mean(train$slow),"\n")
cat("Validation slow rate:",mean(validation$slow),"\n")
```

Executed R output

```
              problem training_n threshold_hours fallback
1 AHV Inspection Unit          1        66.86236     TRUE
2           APPLIANCE        156       887.23875    FALSE
3      Abandoned Bike         24        66.86236     TRUE
4   Abandoned Vehicle        576         5.45194    FALSE
5         Air Quality         76        69.66667    FALSE
6    Animal in a Park         61         4.15667    FALSE
Global training q75 hours: 66.8624
Training slow rate: 0.252062
Validation slow rate: 0.261743
```

24. Logistic Regression Validation Performance

Build the logistic regression model using the training set. Then use the model to predict slow requests in the validation set.

What probability cutoff did your team use to classify requests as slow?

[ ] 0.50 [X] Other cutoff: 0.25

If you used a cutoff other than 0.50, explain why.

The 0.25 cutoff was set before fitting as a screening-oriented choice for an outcome near 25% prevalence. It is not an optimized business cutoff because error costs and queue capacity are unknown. The 0.50 comparison flags no records, so its 73.83% accuracy hides zero recall. At 0.25, 44.99% of validation requests are flagged.

Complete the confusion matrix for your validation set.

	Predicted Not Slow	**Predicted Slow**
Actually Not Slow	4,997 (TN)	3,789 (FP)
Actually Slow	1,550 (FN)	1,565 (TP)

Record your classification performance measures.

Metric	**Validation value at 0.25**
Accuracy	55.14%
Sensitivity / Recall	50.24%
Specificity	56.87%
Precision	29.23%

Metric	Validation value at 0.25
F1	0.3696
ROC AUC	0.5542
Flagged share	44.99%

Accuracy measures the overall percentage of correctly classified requests. Sensitivity measures the percentage of actual slow requests that were correctly flagged. Specificity measures the percentage of actual not-slow requests that were correctly identified.

Which performance measure is most important for this business problem? Explain.

Sensitivity is important because missed slow requests lose an opportunity for early follow-up. It is 50.24%, so about half are still missed. Precision (29.23%) and flag volume are also essential: 3,789 false alerts create workload. Accuracy alone is misleading; AUC is only 0.5542, indicating weak ranking ability.

R code for question 24

```
logistic <- glm(reformulate(predictors,"slow"),
                data=train, family=binomial())
prob <- predict(logistic, newdata=validation,
                type="response")
print(classification_metrics,row.names=FALSE)
cat("Validation AUC:",auc,"\n")
```

Executed R output

```
 cutoff   TN   FP   FN   TP accuracy sensitivity specificity precision
   0.25 4997 3789 1550 1565 0.551382    0.502408    0.568746  0.292305
   0.50 8786    0 3115    0 0.738257    0.000000    1.000000       NaN
       F1 flag_rate
 0.369583  0.449878
 0.000000  0.000000
Validation AUC: 0.554211
```

25. Logistic Regression Interpretation

What types of requests were more likely to be classified as slow?

At cutoff 0.25, 99.43% of Bronx validation requests are flagged, compared with 20.94% in Brooklyn and 1.17% in Staten Island. Phone submissions are flagged 50.70% versus 39.03% online. These are model flag rates, not causal effects or proof of fair treatment. The borough imbalance is a major reason to avoid automatic triage.

Which error is more concerning for this business problem?

[ ] Predicting slow when not slow [X] Predicting not slow when actually slow

Explain your answer.

A false negative is more concerning for the screening objective because an actually slow request receives no early flag. There are 1,550 false negatives. However, the 3,789 false positives would divert staff time, so false-positive costs and capacity must also shape any future cutoff.

R code for question 25

```
print(read.csv("analysis/tables/validation_risk_borough.csv"),
      row.names=FALSE)
print(read.csv(
 "analysis/tables/validation_risk_open_data_channel_type.csv"),
 row.names=FALSE)
```

Executed R output

```
      category    n actual_slow mean_probability flag_rate
      BROOKLYN 3748    0.224653         0.228750 0.2094450
```

```
         BRONX 2288    0.335664         0.313694 0.9943182
     MANHATTAN 2480    0.247984         0.243244 0.3737903
        QUEENS 2958    0.269777         0.249097 0.4604462
 STATEN ISLAND  427    0.215457         0.196246 0.0117096
 category    n actual_slow mean_probability flag_rate
   ONLINE 5081    0.256446         0.245057  0.390278
   MOBILE 2534    0.267956         0.257378  0.482242
    PHONE 3361    0.271050         0.257890  0.506992
  UNKNOWN  925    0.240000         0.253899  0.481081
```

Part 8: Business Recommendation

26. Main Findings

Summarize your three most important findings.

Finding	Evidence
Duration varies widely	Median 6.68 h versus mean 251.36 h; problem/agency differences
Log fit does not ensure hour accuracy	Log RMSE 1.3089; hour RMSE 1,373.18 versus baseline 970.97
Slow classification is weak	AUC 0.5542; 50.24% recall, 29.23% precision; large borough imbalance

R code for question 26

```
cat("Median/mean hours:",median(y),mean(y),"\n")
print(linear_metrics[2,],row.names=FALSE)
print(classification_metrics[1,],row.names=FALSE)
```

Executed R output

```
Median/mean hours: 6.68167 251.356
        set   RMSE      MAE R_squared
 Validation 1.3089 0.927796  0.658189
 cutoff   TN   FP   FN   TP accuracy sensitivity specificity precision
   0.25 4997 3789 1550 1565 0.551382    0.502408    0.568746  0.292305
       F1 flag_rate
 0.369583  0.449878
```

27. Recommendation

What should city operations leaders do based on your analysis?

Do not deploy the current classifier for automatic prioritization. Use the descriptive results to investigate long recorded closure workflows and missing timestamps. If predictions are piloted, run them in shadow mode with staff review, compare against simple operational rules, and test a later-month sample including open requests. Set a review-capacity limit and explicit costs before selecting a cutoff.

R code for question 27

```
cat("Flags per 1000:",round(1000*mean(prob>=.25)),"\n")
cat("Correct flags per 1000:",
    round(1000*sum(prob>=.25 & validation$slow==1)/
          nrow(validation)),"\n")
```

Executed R output

```
Flags per 1000: 450
Correct flags per 1000: 132
```

28. Responsible Use

What should leaders be careful about if they use this model?

A slow prediction does not measure urgency, harm, or service eligibility. Keep safety and established service obligations ahead of the model. Monitor recall, false-positive rates, and workload by borough and problem type. Closure timestamps may reflect administrative practices. Do not infer that borough causes delays or that closed means the problem was fixed.

R code for question 28

```
print(read.csv("analysis/tables/optional_borough_monitor.csv")[,
 c("borough","n","sensitivity","specificity","flag_rate")],
 row.names=FALSE)
```

Executed R output

```
       borough    n sensitivity specificity flag_rate
      BROOKLYN 3748   0.2232779  0.79456297 0.2094450
         BRONX 2288   0.9934896  0.00526316 0.9943182
     MANHATTAN 2480   0.3707317  0.62520107 0.3737903
        QUEENS 2958   0.4824561  0.54768519 0.4604462
 STATEN ISLAND  427   0.0108696  0.98805970 0.0117096
```

29. Limitations

Identify at least two limitations of your analysis.

Limitation	Why it matters
One supplied month and unknown sampling method	Cannot generalize to other months or all city requests; boundary counts uneven
All records marked Closed	Still-open requests may be absent; 332 lack closure timestamps
Random within-month holdout	Does not test future periods or correlated related requests
Administrative closure outcome	Does not demonstrate issue resolution or resident satisfaction
Relative threshold and sparse-type fallback	Not an SLA; some rare types use a less comparable global cutoff
Weak classification and geographic imbalance	Automatic prioritization could allocate attention poorly

R code for question 29

```
print(table(raw$status))
print(range(d$created))
cat("Excluded unlabeled durations:",sum(reason!="Retained"),"\n")
```

Executed R output

```
Closed
 40000
[1] "2025-04-01 09:43:16 EDT" "2025-04-30 09:40:54 EDT"
Excluded unlabeled durations: 332
```

Part 9: AI and Tool Use Reflection

30. Tool Use

Did you use AI or other tools to support your work?

[ ] No [X] Yes

If yes, briefly describe how you used the tool and how you checked the output.

Yes. Codex AI helped inspect the assignment, draft base-R code, interpret outputs, and assemble the worksheet and presentation. The analysis was executed in R 4.6.1. Automated checks verified nonoverlapping splits, valid targets, complete predictions, full model rank, logistic convergence, and confusion-matrix totals. All numeric results come from saved R outputs; document assembly used Python. The student should rerun the script and review the interpretations before submission.

R code for question 30

```
cat(R.version.string,"\n")
cat(readLines("analysis/verification.txt"),sep="\n")
```

Executed R output

```
R version 4.6.1 (2026-06-24 ucrt)
All R invariants passed.
Input MD5: e83fa732cededd45cf760884b1a8f0a1
Linear model rank: 62
Logistic converged: TRUE
```

31. Learning Reflection

What is one data preparation or modeling decision you understand better after completing this project?

A key modeling lesson is that a transformation changes what prediction error means. log1p kept zero-hour observations and improved prediction on the transformed scale, but converting predictions back to hours did not beat the mean baseline on RMSE. This shows why I need to connect evaluation units to the business decision, rather than treating a good transformed R-squared as a promise of accurate completion times.

R code for question 31

```
print(hours_metrics,row.names=FALSE)
cat("Compare errors on the scale needed for the decision.\n")
```

Executed R output

```
                    model     RMSE     MAE    R_squared
 Smearing adjusted linear 1373.178 316.646 -1.00009e+00
   Training mean baseline  970.967 392.307 -1.31544e-05
Compare errors on the scale needed for the decision.
```

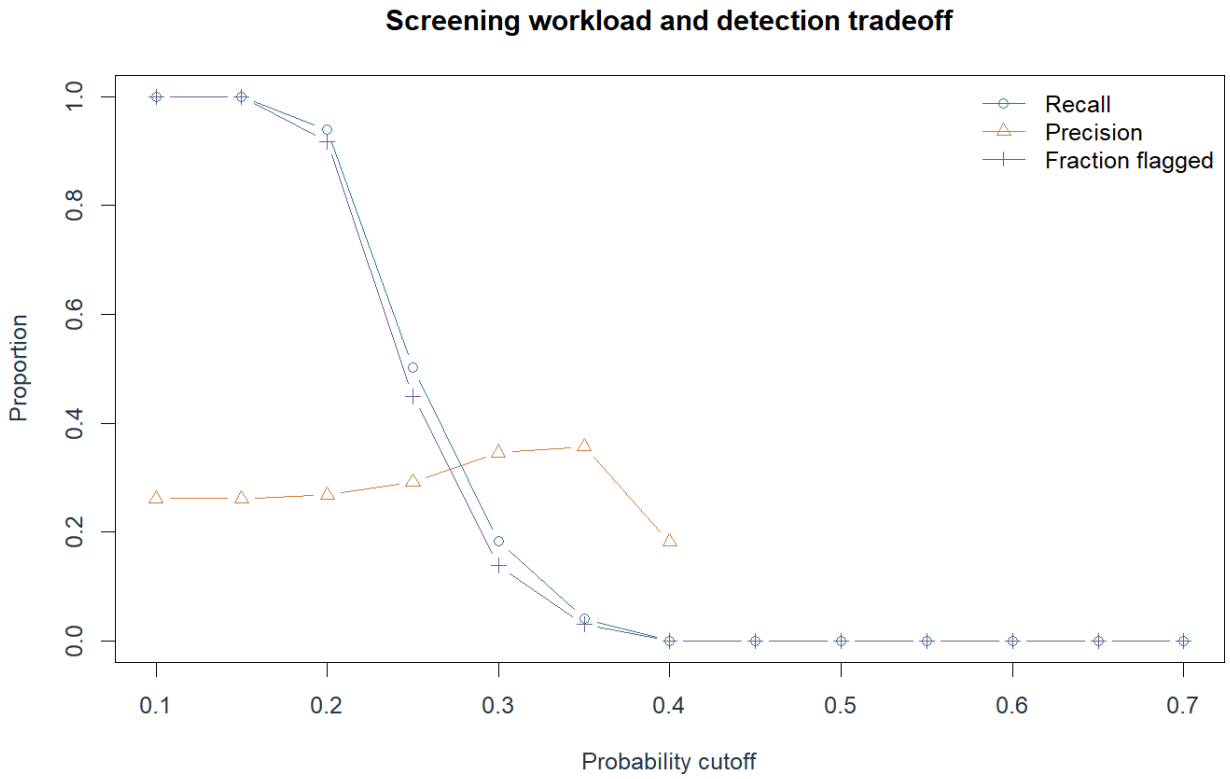
Reminder: Your worksheet should explain your reasoning. Do not simply paste code output without interpreting what it means for the project.

Optional LinkedIn Article Analysis

Question: How does the probability cutoff change detection and staff workload, and does the same cutoff behave similarly across boroughs?

R code: run the Optional LinkedIn Article Analysis section of NYC311_Analysis.R. The following examples inspect its executed results.

```
read.csv("analysis/tables/optional_threshold_tradeoff.csv")
read.csv("analysis/tables/optional_borough_monitor.csv")
```



At 0.25, the model flags approximately 450 of every 1,000 requests, with about 132 correct flags. It misses about half of actual slow requests. At 0.50 it flags none. A lower cutoff can increase recall while creating more review work; no optimal cutoff can be claimed without costs and capacity. These validation comparisons are exploratory, not an independent test of a tuned policy.

The same cutoff flags 99.43% of Bronx validation requests but only 1.17% in Staten Island. Recall is 99.35% and 1.09%, respectively. Different request mix and group calibration may contribute; these results do not establish discrimination, causality, or operational fairness. They show why a model can require subgroup review even when its overall metrics are available.

Sources and data definitions

Supplied sources: Project Overview- NYC 311 Service Requests.pdf; Project 1 Student Worksheet.docx; NYC311_April2025_RawSample.csv. No separate data dictionary or lecture script was included. The field-use table in question 5 documents the operational interpretation used for this analysis.

NYC 311 Reporting FAQ: https://www.nyc.gov/site/311reporting/faq/faq.page (accessed September 26, 2026). Defines the duration as time between receipt and agency-recorded closure.

NYC311 Service Requests: https://portal.311.nyc.gov/article/?kanumber=KA-03116 (accessed September 26, 2026). A closed service request may still have an unresolved underlying issue.

NYC Open Data dataset and column definitions: https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9 (accessed September 26, 2026). The supplied sample is the analysis input; the live dataset was not substituted.